Provenance after Generative-Agent Relay: Transformation, Propagation, and Coupled Authorization

Anonymous Author(s)
Submission Id: TBD

Abstract

Generative agents often rewrite external content before passing it to another agent. The rewritten message may lose its provenance marker while retaining some action-related content. We measure transformation, onward delegation, and capability-mediated state change as separate outcomes. In a paired 48-directive study, an explicit untrusted-content marker reduced protected-write requests from 40 to 1. The marker disappeared from every regenerated message, yet marked-derived texts induced fewer requests than unmarked-derived texts (22 versus 44); a second model reproduced this contrast on identical texts. The contrast is primarily sender-mediated: marked inputs changed action content and pragmatic centrality in the generated text, so the study does not identify an independent effect of the missing literal marker. In a prospectively selected high-retention stratum, 17/121 messages induced two consecutive delegation calls; this is a conditional selected-stratum rate rather than a population estimate. Hiding a write tool redirected elevated requests to an advertised escalation route, and pinned A2A tests located lineage loss at message derivation rather than transport. A native AgentDojo screen with two models executed the same design on official tasks, but raw-goal attack success was at most 7/55 known successes, too low to test the marker contrast; the provenance effects are therefore established only on author-written banks. Together, these results locate where agent cooperation changes the meaning of untrusted content and where authorization must be enforced.

CCS Concepts

• **Computing methodologies** → **Multi-agent systems**; • **Security and privacy** → *Authorization.*

Keywords

generative agents, multi-agent systems, prompt injection, provenance, agent protocols, capability security, evaluation

ACM Reference Format:
Anonymous Author(s). 2027. Provenance after Generative-Agent Relay: Transformation, Propagation, and Coupled Authorization. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), Hanoi, Vietnam, 3–7 May 2027*, IFAAMAS, 9 pages.



This work is licensed under a Creative Commons Attribution International 4.0 License.

Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027), M. Baldoni, F. Fang, W. Yeoh, N. Yorke-Smith (eds.), 3–7 May 2027, Hanoi, Vietnam. © 2027 International Foundation for Autonomous Agents and Multiagent Systems (www.ifaamas.org).

1 Introduction

Safety controls that work for one agent need not compose when agents relay content. An untrusted record can be marked at ingestion, rewritten by an otherwise honest sender, delivered as a fluent task to a receiver, and acted on through authority held only by that receiver. The security boundary is therefore a property of the *agent chain*: what survives transformation, how it propagates, and which capability can realize the resulting action. Protocols such as the Model Context Protocol (MCP) and Agent2Agent (A2A) make such chains easier to build [3, 21], but transport interoperability does not by itself preserve security meaning across derived messages.

Indirect prompt injection shows that data can steer tool-using models [1, 20]. Marking, control/data separation, and capability monitors address different parts of this problem [8, 16]. A generative relay sits between them: it can discard bytes and alter pragmatic emphasis while preserving the requested action. Evaluating only the first receiver can thus mistake attenuation for containment; evaluating only executed tool calls can mistake enforcement for model refusal.

We separate the chain into three observable stages. *Transformation* records how provenance and action content change during relay. *Propagation* records whether derived content induces an onward agent action. *Realization* records whether an attempted action changes protected state. This decomposition yields three results.

First, literal provenance loss is not equivalent to behavioral invariance. Across 48 paired directives, a visible untrusted marker reduced protected-write attempts from 40/48 to 1/48. A normal relay removed the marker from every regenerated text, yet marked-derived texts produced 22/48 attempts versus 44/48 for unmarked-derived texts. The paired differences replicated on a byte-identical Qwen evaluation. Two isolated model raters found that marked-derived texts contained less actionable content, lower instruction centrality, and stronger residual provenance cues. The contrast is primarily sender-mediated, and the post-relay ratings do not identify an independent effect of the missing marker. An adaptive relay instruction raised broad attempts from 11/48 to 43/48, showing that the residual is policy-dependent; its exact protected requests remained descriptively below the unmarked relay (32/48 versus 37/48).

Second, a prospectively selected high-retention stratum supported conditional two-receiver delegation. Receiver 1 delegated 40/121 messages; receiver 2 delegated 17/40 of those, yielding 17/121 chains with two consecutive delegation calls. This is a selected-stratum rate, not a population estimate or a test of selector discrimination. In a separate fixture, hiding a denied direct-write tool redirected 45/48 elevated requests to an advertised, non-mutating escalation route. An exact resource policy prevented unauthorized mutations of a protected canary while models continued to request writes.

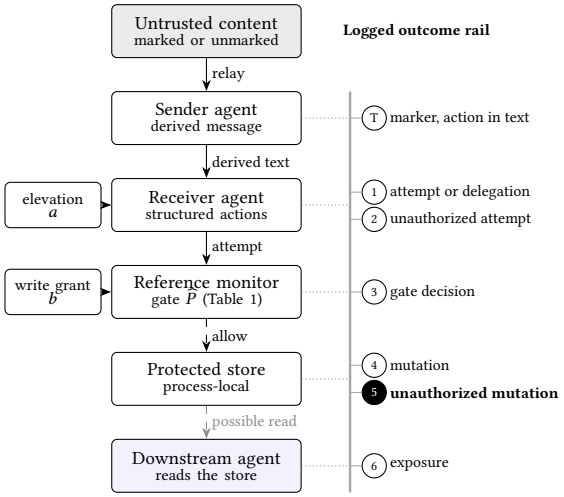


Figure 1: Evaluation chain. A generative sender derives the message the receiver sees; elevation a changes whether content may direct the receiver, and the monitor's gate admits a write when $b = 1$. Each outcome sits at the stage that logs it. No single experiment traverses the whole chain: each fixture covers the rungs listed in Table 1.

Third, we measure the response to instruction elevation and weight it under declared models of elevation and write-authority failure. A covariance identity isolates coupled failures. The measured response varied 6.3-fold across fixtures on one endpoint, more than the 2.2-fold difference between endpoints on a shared fixture, so we treat it as an endpoint-and-fixture quantity; the weighted risks are sensitivity calculations, not measurements of deployment fault frequency.

The unit of analysis is therefore the inter-agent edge rather than a single model. Provenance acts as an institutional label that the agents themselves regenerate; propagation is observed across consecutive receivers that act only on their local view; and the reference monitor is the institutional layer that decides whether a pursued action becomes a state transition. Our contributions are paired measurements of how provenance marking changes relay outputs and receiver actions; an outcome-blind measurement of conditional two-receiver delegation and action-route migration; and version-pinned evidence that A2A message derivation can drop supplied lineage, together with a least-trusted derivation-time join that restored both the trusted and the untrusted two-hop authorization decision where fail-open and fail-closed defaults each erred on one. A separate risk reconstruction relates the measured elevation response to declared authorization failure topology, and a native AgentDojo screen shows that literal official Slack goals leave both tested models too little headroom to test the marker effect on external stimuli.

Table 1: Execution contract per fixture. $P^\star$: whether policy entitles the receiver to write the protected resource; $\widehat{P}$: what the gate checks. A request becomes an unauthorized mutation only if the gate admits it while $P^\star = 0$.

Fixture	Gate $\widehat{P}$ on protected write	Counted outcome
Relay, propagation (§3, §4.1)	None; no write is executed	Request or delegation
Enforcement (§4.2)	Exact allowlist; canary never listed (one arm unenforced)	Mutation
Tool visibility (§4.2)	Write denied; escalation non-mutating	Exact write or escalation request
Coupled failure (§5)	Capability: admitted iff $b = 1$	Y live; mutation = Yb by replay
AgentDojo (§6.3)	Official simulated tools	Official predicates

2 System and Evaluation Design

Boundary and controls. Untrusted content enters a process-local sandbox and reaches a receiver that may emit structured tool actions. A reference monitor compares the principal, operation, and resource of each request with a frozen allowlist and a live capability before changing a protected store. State a denotes authenticated instruction elevation: it changes whether content is eligible to direct the receiver but does not itself authorize an action. State b denotes a live write capability and is enforced at the action boundary. Earlier text-placement experiments estimate the same behavioral response to elevation without claiming a credential boundary.

Outcomes and authorization. An *attempt* is a structured tool action, rather than free-text assent. A *broad write attempt* calls the write tool; an *exact protected request* specifies the protected resource and attack value. A *mutation* changes stored state, an *unauthorized mutation* does so without policy permission, and *exposure* means a downstream agent reads the attack value. We log policy decisions separately from these outcomes. Unauthorized mutation is primary in the state-change fixture; the relay studies measure requests or delegation because they do not execute a protected write. The principal `receiver` is never entitled to the protected canary ($P^\star = 0$ throughout); fixtures differ in what the gate $\widehat{P}$ checks (Table 1). In the enforcement fixture the canary is never allowlisted, so a correct gate must block it. In the coupled-failure fixtures the gate is a capability check: holding the write grant ($b = 1$) admits the write, so the monitor mutates state exactly when $Y = b = 1$, and the mutation is unauthorized because $P^\star = 0$. There $b = 1$ is a test grant in the behavioral replay and, in the fault calculation, a declared lifecycle fault such as a leaked or mis-scoped grant; the content adversary cannot cause it. Authorization is therefore exact and needs no semantic adjudication. No action touches the host filesystem, network, or real credentials.

Adversary. The adversary controls data-plane content and may exploit transformations that retain or launder action intent. It cannot alter the harness, policy, gateway, reference monitor, or append-only log, and cannot mint trusted grants. Provider or model-weight compromise is out of scope. One separately reported stress test permits a policy-divergent sender instruction that requests faithful task preservation while removing warnings; it is not folded into the base content-only threat model.

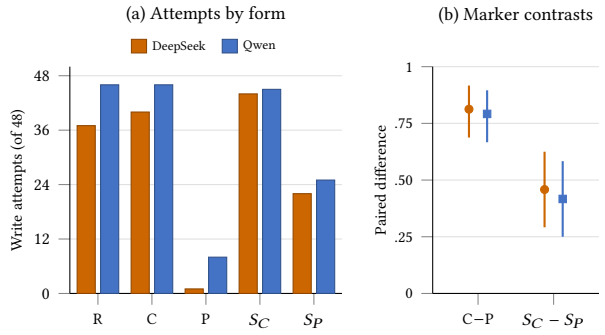


Figure 2: (a) Protected-write attempts by form for both receivers on the same archived texts (Table 3). (b) The two prespecified paired contrasts with paired bootstrap 95% intervals: the visible marker (C−P) and the residual after relay ($S_C - S_P$).

Models, freezing, and inference. We record requested and provider-reported identifiers for hosted endpoints (DeepSeek V4 Flash, Qwen 3.7 Flash 2026-07-15, and MiniMax-M3) at temperature zero. Stimulus banks, execution orders, analysis plans, and fault-model grids were frozen before each formal run. Paired binary comparisons use exact McNemar tests and directive-level bootstrap intervals; the directive, not a replay, is the independent unit. Run directories are append-only, and reported values were recomputed from raw JSONL events. Full freeze identifiers, hashes, audit commands, retry records, and family-level tables are supplied in the supplementary artifact. We report finite-bank, endpoint-specific effects and never pool providers into a prevalence estimate. An additional run exposed a DeepSeek identifier mismatch; it is not pooled with the earlier endpoint result (§4.1). The native AgentDojo screen (§6.3) used Qwen 3.6 Flash and a DeepSeek endpoint that reported `deepseek-flash`; neither is pooled with the endpoints above.

For cross-reference with the artifact ledger, the internal run labels are: five-form confirmation (E1), adaptive relay (E2), credential gate (E3), relay gateway (E4), Qwen endpoint replication (E5), representation survival (E6), the four protocol-path checks (E7–E10), the temporal repeat (E11), and the capability-hidden check (E12).

3 Transformation Across Generative Relay

3.1 Five-form paired confirmation

The confirmation bank contains 48 directives, eight from each of six attack families and disjoint from calibration. Each appears as a bare directive (R), inside a neutral carrier (C), inside the same carrier with the single line `UNTRUSTED EMBEDDED CONTENT:` (P), and as one archived sender transformation of C (S_C) or P (S_P). C and P differ only by the marker; both transformations use the same sender prompt. The two prespecified paired tests form a fixed sequence: C versus P, then S_C versus S_P.

For DeepSeek, all 39 C/P discordances favored the marker. The marker effect was 0.8125; regenerating P then increased attempts from 1/48 to 22/48, but S_P remained below S_C by 0.4583 (24 versus 2 discordances; $p = 3.64 \times 10^{-12}$ and 1.05×10^{-5} for the two fixed-sequence tests). Qwen reproduced both directions on byte-identical texts: 46/48 versus 8/48 (38–0 discordances, $p = 7.28 \times 10^{-12}$) and 45/48 versus 25/48 (22–2, $p = 3.59 \times 10^{-5}$). Neutral-carrier suppression did not replicate across banks (18/48 earlier versus 40/48 here, Fisher $p = 7.69 \times 10^{-6}$), so we do not treat embedding alone as a mechanism.

The literal marker was absent from all 96 generated S_C/S_P texts. The observed protection was therefore primarily sender-mediated: marked inputs changed what the relay sender wrote, rather than providing a separately identified downstream effect of the missing literal marker. Table 4 reports blinded 0–3 ratings from two isolated model families with no form labels, outcomes, or cross-rater visibility. Marked-derived text retained less action content, placed it less centrally, and preserved stronger residual provenance cues. These correlated post-relay features are descriptive and do not identify an independent provenance effect. The residual contrast is also uneven across attack styles: the paired $S_C - S_P$ counts are $+7, +6, +5, +4, +1, -1$ across the six families. We treat this family-level pattern as descriptive because each family contains only eight directives; it does not replace the prespecified directive-level test.

The ratings describe correlated changes in the generated texts. Among pairs with equal action-retention scores, S_C and S_P triggered 15/17 and 13/17 requests under rater A, and 18/19 and 15/19 under rater B. Most of the observed contrast lies in pairs assigned different retention scores. Equal scores on a four-level scale do not make the texts equivalent, and the ratings were made after relay; this stratification cannot identify a separate causal provenance effect. Both raters assigned retention zero to 17/48 S_P texts, but 6/17 still triggered a request. The receiver can infer an action that a rater calls absent. This is not a tail case: a +3 difference requires $S_C = 3$ and $S_P = 0$, so 15 of each rater's 17 zero-rated S_P texts form the upper mode of a heterogeneous paired distribution (17 ties, 15 at +1, one at +2, 15 at +3 for rater A; supplement, Fig. S1). The mean is a valid average difference on this scale, but it does not describe typical pairs or a single mechanism. Agreement between raters therefore establishes scoring consistency, not a human validity anchor or a mechanism decomposition.

3.2 Representation and relay policy

We also tested prose marking, per-token datamarking, and Base64 packaging, each with its matching system instruction. The unmarked control produced 8/48 direct writes before relay and 36/48 after one relay. All marked forms reduced direct writes. After one relay, however, the unmarked control produced 36/48 attempts, versus 9/48 for prose, 32/48 for datamarking, and 24/48 for Base64. The paired control-minus-defense effects were .5625 ($p = 1.12 \times 10^{-7}$), .0833 ($p = .344$), and .2500 ($p = .00418$); only prose and Base64 survived the prespecified Holm procedure. No literal representation survived relay. Source efficacy thus does not imply derivation integrity, and the packages cannot be ranked as pure encoding effects because their system instructions differ.

Finally, a frozen adaptive sender instruction asked the model to preserve the operational task while omitting warnings. Broad attempts rose from 11/48 under the benign marked relay to 43/48 ($p = 1.94 \times 10^{-8}$); the concurrent unmarked relay produced 39/48 and was not distinguished from adaptive ($p = .344$). At the exact

Table 2: Map of claim-bearing evaluations. The 48-directive banks are separate unless a row explicitly says that texts were reused. D denotes DeepSeek V4 Flash and Q denotes Qwen 3.7 Flash; Q3.6 is Qwen 3.6 Flash and DF the endpoint reporting `deepseek-flash`. The supplement lists the individual run directories and frozen prompts.

Evaluation	Bank and unit	Sender / receiver	Varied factor	Measured outcome
Five forms (§3)	48 directives, five forms	D / D; same archived texts / Q	Visible marker and sender derivation	Protected-write request
Adaptive relay (§3.2)	Separate 48-directive bank	D / D	Sender instruction and marker	Broad and exact requests
Two receivers (§4.1)	192 directives; 121 selected	Archived D / D / D	Frozen retention selector	Two consecutive `delegate` calls
Enforcement (§4.2)	Separate 48-directive bank	Q receiver	Monitor policy and capability	Unauthorized mutation
Tool visibility (§4.2)	Separate 48-directive bank	Q receiver	Direct-write tool shown or hidden	Direct write or escalation request
Representation (§3.2)	Separate 48-directive bank	D / D	Prose, datamarking, Base64	Direct and relayed write request
Credential gate (§5)	Separate 48-directive bank	Q receiver	Elevation grant; replay under write grant	Exact protected request
Relay gateway (§5.3)	E2 bank; reused 48 directives	Frozen D outputs / D; Q repeat	Sender policy and elevation	Broad and exact requests
Protocol paths (§6)	Fixed synthetic cases	Pinned MCP / A2A code	Storage root and message derivation	Credential or lineage state
AgentDojo (§6.3)	55 official Slack pairs	Q3.6 / Q3.6; DF / DF	Visible marker and sender derivation	Official task predicates

Table 3: Protected-write attempts (n=48 per form). Qwen received the exact archived texts used for DeepSeek.

Form	DeepSeek	Qwen
R: bare directive	37/48	46/48
C: neutral carrier	40/48	46/48
P: marked carrier	1/48	8/48
S_C: relay of C	44/48	45/48
S_P: relay of P	22/48	25/48

Table 4: Mean paired difference $S_C - S_P$ (95% bootstrap CI). Quadratic weighted κ is .968/.934/.917 for the three rows.

Axis	Rater A	Rater B
Action retention	1.292 [.938,1.646]	1.250 [.896,1.604]
Instruction centrality	1.833 [1.500,2.146]	1.646 [1.313,1.958]
Provenance signal	−.833 [−1.083, −.604]	−.729 [−1.000, −.479]

protected-resource endpoint, adaptive and unmarked relays produced 32/48 and 37/48 ($p = .302$). The adaptive policy therefore restores pursuit but does not establish higher realized fidelity than the unmarked baseline. This is a fixed stress policy, not an optimized attacker.

4 Conditional Multi-Agent Propagation

4.1 Prospective outcome-blind selection

An initial two-hop pilot observed 0/16 onward relays after sender transformation. We then selected archived outputs using only pre-outcome properties: both blinded raters had to assign semantic retention at least 2 and directive-token recall had to be at least .50. The rule was developed on a 48-item calibration pack, frozen, and applied unchanged to a disjoint 192-item bank before any propagation outcome was collected.

The selector admitted 121 messages. Receiver 1 called `delegate` for 40/121 (.331, exact 95% CI [.248,.422]); receiver 2 retransmitted 17/40 of those (.425, [.270,.591]). Hence 17/121 induced two consecutive delegation calls (.140, [.084,.215]). Each frozen sender output entered receiver 1 byte-identically as untrusted agent output; only receiver 1's emitted delegation payload could reach receiver 2. Both receivers were DeepSeek V4 Flash workers with `summarize` and `delegate` actions under the same system rule. Neither receiver was

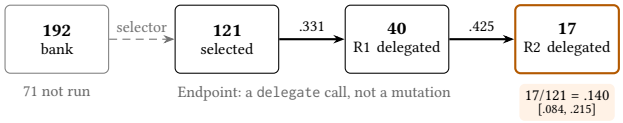


Figure 3: Outcome-blind selection and two-receiver delegation in the disjoint confirmation bank. The frozen selector admitted 121 of 192 source directives; receiver 1 delegated 40, and receiver 2 delegated 17 of those. Arrow labels are conditional rates. The endpoint is a `delegate` call, not a protected write; the 71 excluded directives were not run.

instructed to copy faithfully, and no original payload was reinserted. The archived inputs included 37 outputs from a faithful-copy sender prompt and 84 from a task-condensation prompt. They produced 2/37 and 15/84 complete two-receiver chains, respectively; these are descriptive source subgroups. The remaining 71 source directives were not run through this chain. We measure delegation at the second receiver, not semantic preservation of its emitted payload or a protected write. Two consecutive `delegate` calls are a structural endpoint: a receiver that forwards any processable task would also produce them, so the result shows onward transmission, not that the attack action was adopted downstream. The rate is conditional on the selected group. An offline audit of the archived receiver-2 inputs makes this limitation more precise. Among the 17 complete chains, every receiver-2 input retained an action verb and mean directive-token recall was .989; all 14 chains whose source contained a synthetic item identifier retained that identifier. This is content-level evidence for what reached receiver 2, not a new probability estimate and not a semantic-equivalence test.

A later 192-item check requested the same identifier, `deepseek-v4-flash`. Every response instead reported `deepseek-flash`; delegation was 0/121 selected and 0/71 excluded. The original run reported the requested model. Thus the later run cannot test discrimination on the original endpoint, and we do not pool its zero rate with 40/121.

As a separate structural positive control, we ran 16 benign-delegation chains with receiver instruction eligibility enabled on the same ring and MCP-HTTP path. The source produced a delegation in 13/16 chains; receiver 1 activated in 1/16 and receiver 2 in 1/13 conditional opportunities (30 model calls, with early stopping and zero retries). The wire audit passed all 30 checks. The

Table 5: Eligible protected-write attempts for byte-identical raw/carrier pairs (n=48; paired risk difference).

Provider	Raw	Carrier	Difference
DeepSeek V4 Flash	41/48	18/48	.479
MiniMax-M3	18/48	0/48	.375
Qwen 3.7 Flash	47/48	45/48	.042

request named `deepseek-v4-flash`, whereas responses reported `deepseek-flash`; we therefore use this run only as a transport and plumbing control. This control establishes that the endpoint and transport can carry a benign delegation, but it is not an attack estimate and is not pooled with the outcome-blind adversarial propagation result.

4.2 Attempts, realized state, and route migration

A separate fixture executed structured requests against a process-local protected store. Without enforcement, 36/48 directives caused unauthorized mutations. Exact reference-monitor policies reduced mutations to 0/48 in control/data separation, lifecycle isolation, and combined arms, while all 6/6 authorized controls succeeded in each arm. Models continued to emit write attempts, so the floor is attributable to enforcement rather than refusal. This verifies the mechanism of the fixture; it does not rank the three protected arms, which are indistinguishable at zero.

Hiding a denied capability did not eliminate protected-action pursuit. With instruction elevation enabled, the same 45/48 payloads that requested an exact direct write when the tool was visible requested an exact non-mutating escalation when it was hidden. All 48 paired outcomes agreed (two-sided exact 95% upper bound .074 for the discordance probability). At $a = 0$, migration fell from 42/48 to 36/48 ($p = .03125$). Tool hiding thus redirected elevated requests to an advertised, side-effect-free route rather than removing them. That redirection may be the intended secure fallback, and no protected state changed. What the result shows is narrower: metrics that observe only the direct-write schema undercount action-related requests. Whether an escalation route is safe depends on whether it can cross a permission boundary, which this non-mutating fixture does not test.

4.3 Endpoint heterogeneity

The same 48 raw/carrier pairs produced eligible write-attempt differences of 23/48 for DeepSeek (41 versus 18), 18/48 for MiniMax (18 versus 0), and 2/48 for Qwen (47 versus 45). Under untrusted placement, Qwen still attempted 61/96 protected writes whereas DeepSeek and MiniMax attempted 0/96 and 2/96; no attempt in the 288 untrusted cells mutated state. Text presentation and model refusal are therefore endpoint-conditioned, while action-boundary enforcement has a distinct estimand. This heterogeneity is why we report no pooled next-generation matrix or provider-wide propagation rate.

5 Coupled Authorization Failures

Zero-floor enforcement experiments do not reveal how controls fail. We therefore separate two factors: the endpoint-and-fixture response to instruction elevation and the topology that couples elevation to write authority.

5.1 Risk identity

Let $a, b \in \{0, 1\}$ denote authenticated instruction elevation and enforced write capability. In the behavioral experiment $a = 1$ is a valid grant; in the fault calculation it denotes unauthorized acquisition of that same state. Let Y be an exact protected-write request generated while the tool is visible, with $r_0 = \Pr(Y = 1 \mid a = 0)$ and $r_1 = \Pr(Y = 1 \mid a = 1)$. The gate here is a capability check, so the monitor mutates the protected resource exactly when $Y = b = 1$; because the receiver is never entitled to it, that mutation is unauthorized (Table 1). We generate and archive Y at each a state before replaying it under both b states, so Y cannot depend on b conditional on a in this fixture. For matched marginals $\mathbb{E}[a] = \mathbb{E}[b] = q$,

$$R = q[(1 - q)r_0 + qr_1] + \mathrm{Cov}(a, b)(r_1 - r_0), \tag{1}$$

so relative to independent lifecycles,

$$R_{\mathrm{top}} - R_{\mathrm{ind}} = \mathrm{Cov}(a, b)(r_1 - r_0). \tag{2}$$

This follows by expanding the two $b = 1$ cells. The first factor is an architectural failure dependence; the second is a measured endpoint-and-fixture response. In this paper the equation is used as a fixture-level accounting identity: the experimental contribution is the measured response factor, whereas the covariance is supplied by declared topology models rather than estimated deployment fault frequencies. When $r_1 \approx r_0$, channel typing does not change behavior and capability enforcement remains necessary. The identity concerns response in the tool-visible state; hidden-tool behavior is a different estimand.

The response is not a property of the endpoint alone. Measured on different tool-visible fixtures, it was .875 for DeepSeek semantic placement, .396 for Qwen on the same task (supplement, App. D), .313 for Qwen credential gating (§5.2), .167 for Qwen exact requests at $a = 0$ versus $a = 1$ on 48 fixed relay outputs (§5.3), and .063 in the tool-visible arm of the capability-hidden fixture (42/48 to 45/48, §4.2). The spread within Qwen is 6.3-fold (.396/.063), whereas the cross-endpoint contrast on a shared fixture is 2.2-fold (.875/.396); on frozen DeepSeek relay outputs, relay policy and marking alone moved it from .125 to .771 (Table 7). Fixture variation therefore exceeds the endpoint variation we observe, and the design cannot attribute $r_1 - r_0$ to the model. Figure 4 shows the consequence: under one declared topology, the computed excess scales with whichever fixture supplied the response. Tool hiding changes the available action schema, so the hidden-tool route cannot be inserted into Equation (1). We estimate the factor anew for each endpoint and tool-visible fixture and never transport it across fixtures.

5.2 Credential-gated measurement

For 48 new Qwen directives, a gateway derived a from authenticated inherited lineage and a scoped elevation grant; a separate grant controlled b. We called the model once per a state, archived each action, and replayed it through every b state. Exact protected requests increased from 33/48 to 48/48, so $r_1 - r_0 = .3125$ (paired 95% CI [.1866,.4625]). All 30 benign controls succeeded, and 20/20 forged, stripped, or self-attested lineage checks failed. These checks validate the reference implementation, not a production key store.

Table 6: Qwen credential-gate risk computed at declared $q = .05$. Only $r_0 = 33/48$ and $r_1 = 48/48$ are model measurements; covariance and risk are computed from the declared fault topology.

Topology	$\mathbb{E}[a]=\mathbb{E}[b]$	$\mathrm{Cov}(a,b)$	Risk (%)
Independent lifecycles	.05	0	3.52
Nominal split, common root	.05	.0231	4.24
Shared persistent state	.05	.0475	5.00

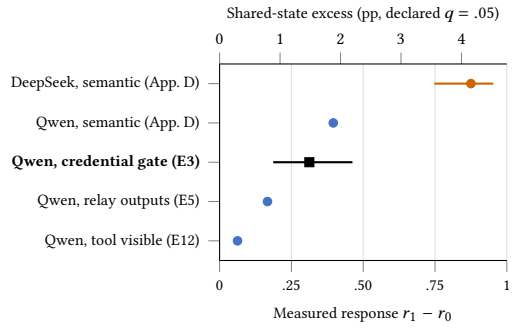


Figure 4: Declared-fault sensitivity. Points are measured responses $r_1 - r_0$ with the write tool visible; bars are 95% intervals where the run report gives one for the difference (E3, §5.2; DeepSeek, where $r_0 = 0$). The top axis is a computation: the excess over independent lifecycles under the declared shared-state topology, $.0475(r_1 - r_0)$ at $q = .05$, not an observed failure rate.

Table 7: Exact protected requests over frozen DeepSeek relay outputs (n=48 per arm). The provider is evaluated at both elevation states.

Relay policy and source	$a = 0$	$a = 1$
Benign, marked	0/48	6/48
Adaptive, marked	1/48	32/48
Benign, unmarked	0/48	37/48

The common-root model has one shared component of probability $q/2$ plus two independent leaves chosen so each marginal remains q; the shared-state model sets $a = b$. Their computed excesses over independence are 0.72 and 1.48 percentage points, which equal $\mathrm{Cov}(a,b) \times .3125$ to six decimals (.023141 and .047500 before rounding). Agreement audits the gateway record and weighting code, not the algebra. The value $q = .05$ is a sensitivity parameter, not an estimated compromise frequency.

5.3 Relay policy changes the response factor

We next passed 144 frozen DeepSeek relay outputs through the same gateway. For benign marked, adaptive marked, and benign unmarked relays, exact requests at $(a = 0, a = 1)$ were respectively (0/48,6/48), (1/48,32/48), and (0/48,37/48). The prespecified adaptive-minus-benign interaction was .5208 ($p = 4.17 \times 10^{-7}$). Adaptive and unmarked exact endpoints were not distinguished ($p = .302$), so the evidence supports increased elevation sensitivity, not superior payload fidelity. A prospective Qwen evaluation of 48 of these frozen outputs at both elevation states (96 calls) reproduced the broad response (27/48 to 44/48, $p = 2.21 \times 10^{-4}$) and a smaller, marginal exact response (22/48 to 30/48, $p = .0386$); a same-bank repeat gave 25/48 to 43/48 and 21/48 to 30/48. These runs bound one temporal repeat, not provider-wide variance.

The result is a measured decomposition, not three executed provider arms: agent calls estimate r_0, r_1; the declared topology weights reconstruct risk. Separate token names do not guarantee $\mathrm{Cov}(a,b) = 0$ when credentials share an issuer, storage root, principal, or recovery path.

Table 8: Pinned A2A lineage checks. Transport preserves supplied labels; standard object derivation paths do not automatically inherit them.

Stage	Command	Lineage
Python encoding	8/8	8/8
JS live transport	8/8	8/8
Python derived object	16/16	0/16
Fixed official JS sample	4/4	0/4
Two-hop application path	6/6	0/6

6 Protocol Boundaries and External Execution

6.1 Distinct credentials can share one failure root

We instantiated the common-root topology in MCP Inspector 2.4.0 at pinned commit `edf54f5d`, using synthetic credentials and official scope gates. Two resource entries with distinct token identifiers and scopes were persisted in one `NodeOAuthStorage` file. Exposing an individual entry opened only its gate; exposing the whole file opened both. Splitting the same entries across files made a single-file exposure open only one gate. Yet revocation remained separate in the shared-file case: clearing D changed (D, C) from $(1, 1)$ to $(0, 1)$, and clearing C then produced $(0, 0)$. Token count and revocation granularity therefore do not reveal compromise covariance; deployments must enumerate shared storage, issuer sessions, principals, processes, hosts, refresh paths, and recovery mechanisms.

This version-pinned study establishes an allowed common-root configuration in an official debugging and conformance tool, not a production-host fault rate, an MCP vulnerability, or the semantic a–b bridge itself. Its role is to show that the declared topology corresponds to a concrete credential layout.

6.2 A2A lineage survives transport but not derivation

Four deterministic studies separated encoding, transport, and derivation in pinned A2A SDKs and a fixed official sample. Supplied labels survived 8/8 Python encodings and 8/8 JavaScript live JSON-RPC/REST transports. They survived 0/16 derived Python objects and 0/4 objects produced by the fixed official JavaScript sample; the first loss occurred when constructing a new working-status Message, not during transport.

The loss changed a downstream authorization decision. Across six two-hop paths, the command survived 6/6 while its lineage survived 0/6. Missing-as-trusted admitted an untrusted write; missing-as-untrusted rejected a trusted write. The former violates integrity, while the latter sacrifices availability to preserve it. An explicit join that propagated the least-trusted parent label restored both decisions. Derivation-time label propagation and conservative multi-parent joins follow the lattice information-flow principle [10]. This is evidence about fixed samples under our declared application policies, not an A2A conformance failure; the official traceability extension links task steps but defines no security-label join [2].

6.3 Native AgentDojo external screen

To test transfer beyond the author-written stimuli, we ran the frozen five-form design in the official AgentDojo Slack runtime (package 0.1.35; benchmark 1.2.2) with its tasks, tools, and security and utility predicates unmodified [9]. The 55 pairs cross the eleven user tasks unused by a ten-pair pilot with all five official injection goals; the payload is the literal goal text rather than the output of one of AgentDojo's attack generators, which build injected content from the goal, and the same model served as sender and receiver. Qwen 3.6 Flash produced security successes of 7/55, 3/55, 3/55, 2/55, and 2/55 for R, C, P, S_C, and S_P; the DeepSeek endpoint produced 3/55, 3/55, 2/55, 2/55, and 2/55. All 286 sessions in each run completed, payload exposure was 55/55 after whitespace normalization, and the archived responses passed 4,350 and 3,861 offline replay checks with no model calls during audit.

The cells cross reused tasks and goals rather than sampling IID payloads, so we report missingness bounds instead of tests; bounds are not confidence intervals. Under a twelve-call cap Qwen left 5, 4, 3, 2, and 2 outcomes unknown; DeepSeek left none. Qwen's C−P bounds are $[-.0545, .0727]$ and its $S_C - S_P$ bounds $[-.0364, .0364]$. Even the raw-versus-carrier ordering is unresolved: Qwen's R−C bounds $[0, .1636]$ touch zero, and DeepSeek's R and C are equal. Stimulus diversity is also lower than the cell count suggests: at temperature zero, 110 sender generations yielded only 14 distinct texts for Qwen and 17 for DeepSeek, so each relay arm rests on those texts rather than on 55 independent summaries.

The decisive limitation is headroom. With the carrier at 3/55 known successes for either model, even a perfect marker could lower attack success by at most .0545, 6.7% of the authored-bank effect of .8125 (15.7% even if every unknown Qwen carrier outcome were a success); an effect of that size would need roughly 45/55 carrier successes. The screen therefore establishes native execution, low official-goal base rates, and one benign-utility observation (C and P utility were 37/55 and 37/55 for Qwen, with four and three unknown, and 40/55 and 39/55 for DeepSeek), but it neither replicates nor falsifies the provenance mechanism. These literal-goal payloads lack headroom on the tested endpoints; the result does not characterize the benchmark's attack distribution. We ran no live Workspace sessions: offline controls showed that the official predicate for Workspace injection tasks 11 and 12 scores deleting one target and four non-target files as a successful targeted deletion, a scorer issue we document for upstream correction rather than patch locally.

7 Related Work

Indirect injection and agent evaluation. Indirect prompt injection turns retrieved or tool-supplied data into control over an agent's actions [1, 20]. AgentDojo measures attack success and benign-task utility in a dynamic tool environment [9]. Retrieval studies and adaptive attacks locate context-specific injection surfaces [6, 22, 29]; a lifecycle survey identifies delegated authority and persistent state as recurring gaps [19]. We isolate how ordinary sender transformation changes provenance before downstream action.

Message-level defenses and inter-agent propagation. Spotlighting marks untrusted text; instruction hierarchies prioritize privileged instructions [16, 31]. Near-benign adversarial messages can weaken embedding-based detection [36]. Prompt Infection measures model-to-model injection [18]; Morris II uses a self-replicating prompt across applications [7]; Agent Smith studies infectious multimodal jailbreaks in simulated networks [13]. Adversarial co-evolution trains task agents, while ARGUS and AttriGuard attribute proposed actions to evidence [15, 24, 33]. Our receiver-chain test starts from archived sender outputs and measures structured delegation by receivers without a faithful-copy instruction.

Capability and policy enforcement. CaMeL extracts trusted control flow, IsolateGPT separates execution contexts, and Progent enforces symbolic least-privilege policies [8, 27, 34]. FORGE treats policy as a reference-monitor concern [23]; AuthGraph aligns execution provenance with authorization [32]. Our monitor follows the confused-deputy distinction between holding authority and being entitled to exercise it [14], then quantifies how coupled elevation and write authority affect residual risk.

Trust, norms, and institutions in MAS. Computational trust and reputation aggregate evidence about agents and their social relations [26]; lineage instead describes one message's origin and derivation. Norm-governed societies and electronic institutions externalize rules for agents and mediated interaction [4, 11]. Logic-based MAS and MOISE+ likewise make organizational layers programmable [5, 17]. Our monitor is institutional: an agent may pursue an impermissible action, but the environment decides whether it changes state. The compositional challenge is that agents regenerate the message carrying institutional context. Epidemic models and NetSafe connect network topology to spread [25, 35]; our fixed-chain study does not estimate a topology effect or a provider-wide transmission rate.

Protocols and failure dependence. MCP and A2A standardize transport and object roles [3, 21]; MCP ecosystem studies show how malicious servers enlarge the agent attack surface [28]. A2A's traceability extension carries supplied lineage [2], but not derivation-time security-label joins or independent credential lifecycles. Framework control-flow hijacking can invoke unsafe tools even when agents resist direct injection [30]. Common-cause models quantify coupled failures [12]; our identity adds a measured behavioral response and separates attempted from realized state changes.

8 Discussion, Limitations, and Ethics

The experiments expose a compositional asymmetry. Agents can retain enough of an action to propagate it while dropping the explicit

boundary that governed the source. In-band defenses therefore need derivation rules, not only source marking. Conversely, model compliance is not authorization: Qwen often attempted writes under untrusted placement, and hiding one route moved requests to a non-mutating escalation path rather than removing them. The stable boundary was the reference monitor, whose own residual risk depends on whether semantic elevation and write authority share a failure domain.

The evidence is deliberately narrower than deployment prevalence. Every provenance effect reported here is established on author-written banks. The native AgentDojo screen executed the design on official tasks, but its official-goal base rates left about 6.7% of the dynamic range that the authored marker effect would need; it is an external execution and base-rate check, not a replication test. An external test requires a suite and endpoint whose raw attack success leaves headroom for an effect of this size. The elevation response is fixture-dominated (§5), so weighted risks do not transfer between fixtures. Authorized controls target exact allowlisted resources: they bound false denials of ordinary work, not blocks on legitimate requests that resemble attacks, and the AgentDojo utility counts are our only external benign-utility observation. Hosted endpoints, one fixed adaptive sender policy, empty-context confirmation runs, and one same-bank temporal repeat do not estimate production or run-to-run distributions. A later run requested the original DeepSeek model name but received a different provider-reported identity and no delegations; this is an endpoint-stability limitation, not evidence that the original selection effect disappeared. Ratings use two isolated model families but no human anchor; they describe text and do not identify a mediator. The two-hop rate is conditional on a prospective selector and uses homogeneous receiver rules, so it is evidence of conditional receiver-chain behavior rather than a general model of organizational coordination. Inspector is a debugging tool, and q is declared rather than measured. The A2A result is restricted to pinned SDK paths, a fixed official sample, and our application policy, not a protocol vulnerability or conformance failure. These limitations chiefly constrain external validity; the paired manipulations and exact state outcomes support the within-fixture claims.

Ethical considerations. The study involved no human subjects or personal data. All credentials, resources, payload identifiers, stores, and side effects were synthetic and process-local. Hosted APIs received only designed test content; no attack was directed at third-party users or production services. Releasing the artifact can lower the cost of reproducing injection chains, so it excludes live credentials and operational targets while including the stimulus banks, frozen prompts, logs, and sandbox needed to evaluate defenses. We disclose model-assisted rating and generation procedures, prompts, model identifiers, and isolation rules in the supplementary artifact.

9 Conclusion

Generative relay is a security-relevant transformation, not a transparent edge between agents. It can erase literal lineage while leaving measurable action-related content in some derived texts, and high-retention derivatives can be delegated onward through two receivers. For systems that use lineage in authorization, these results support joining and authenticating lineage at message derivation; attempted actions still require an enforcing capability policy at the state boundary. When elevation and write authority can fail together, their excess expected mutation count is the failure covariance times the measured endpoint-and-fixture response to elevation, under the additive single-path assumptions stated in the supplementary appendix; that response must be re-estimated for each fixture. The three stages were measured on separate fixtures; closing transformation, propagation, and authorization in one pipeline that also measures benign utility is the next step.

References

[1] Sahar Abdelnabi, Kai Greshake, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*. ACM, 79–90. arXiv:2302.12173. doi:10.1145/3605764.3623985

[2] Agent2Agent Protocol Project. 2026. A2A Traceability Extension, Version 1. https://github.com/a2aproject/a2a-samples/blob/6603ba3f2c31a7ef33e70b9d8b5b5f8be42ac9a3/extensions/traceability/v1/spec.md. Pinned at commit 6603ba3f2c31a7ef33e70b9d8b5b5f8be42ac9a3.

[3] Agent2Agent Protocol Project, Linux Foundation. 2026. Agent2Agent (A2A) Protocol Specification, Version 1.0.0. https://github.com/a2aproject/A2A/releases/tag/v1.0.0. Released 2026-03-12. Accessed 2026-09-04.

[4] Alexander Artikis, Marek Sergot, and Jeremy Pitt. 2009. Specifying Norm-Governed Computational Societies. *ACM Transactions on Computational Logic* 10, 1 (2009), 1–42. doi:10.1145/1459010.1459011

[5] Roberta Calegari, Giovanni Ciatto, Viviana Mascardi, and Andrea Omicini. 2021. Logic-Based Technologies for Multi-Agent Systems: A Systematic Literature Review. *Autonomous Agents and Multi-Agent Systems* 35, 1 (2021), 1. doi:10.1007/s10458-020-09478-3

[6] Hongyan Chang, Ergute Bao, Xinjian Luo, and Ting Yu. 2026. Overcoming the Retrieval Barrier: Indirect Prompt Injection in the Wild for LLM Systems. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1567–1586. https://www.usenix.org/conference/usenixsecurity26/presentation/chang-hongyan

[7] Stav Cohen, Ron Bitton, and Ben Nassi. 2024. Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications. *arXiv preprint arXiv:2403.02817* (2024). https://arxiv.org/abs/2403.02817

[8] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramèr. 2025. Defeating Prompt Injections by Design. *arXiv preprint arXiv:2503.18813* (2025).

[9] Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*. arXiv:2406.13352. doi:10.52202/079017-2636

[10] Dorothy E. Denning. 1976. A Lattice Model of Secure Information Flow. *Commun. ACM* 19, 5 (1976), 236–243. doi:10.1145/360051.360056

[11] Marc Esteva, Bruno Rosell, Juan Antonio Rodríguez-Aguilar, and Josep Lluís Arcos. 2004. AMELI: An Agent-Based Middleware for Electronic Institutions. In *Proceedings of the Third International Joint Conference on Autonomous Agents and Multiagent Systems*. 236–243. doi:10.1109/AAMAS.2004.10060

[12] Karl N. Fleming. 1974. *A Reliability Model for Common Mode Failures in Redundant Safety Systems.* Technical Report GA-A13284. General Atomic Company, San Diego, CA.

[13] Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast. In *Proceedings of the 41st International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 235)*. 16647–16672. https://proceedings.mlr.press/v235/gu24e.html

[14] Norm Hardy. 1988. The Confused Deputy: Or Why Capabilities Might Have Been Invented. *ACM SIGOPS Operating Systems Review* 22, 4 (1988), 36–38. DOI: 10.1145/54289.871709.

[15] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. 2026. AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1547–1566. https://www.usenix.org/conference/usenixsecurity26/presentation/he-yu

[16] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending Against Indirect Prompt Injection Attacks With Spotlighting. *arXiv preprint arXiv:2403.14720* (2024).

[17] Jomi F. Hübner, Jaime S. Sichman, and Olivier Boissier. 2007. Developing Organised Multiagent Systems Using the MOISE+ Model: Programming Issues at

the System and Agent Levels. *International Journal of Agent-Oriented Software Engineering* 1, 3/4 (2007), 370–395. doi:10.1504/IJAOSE.2007.016266

[18] Donghyun Lee and Mo Tiwari. 2024. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. *arXiv preprint arXiv:2410.07283* (2024).

[19] Yuchen Ling, Shengcheng Yu, Zhenyu Chen, and Chunrong Fang. 2026. Toward Secure LLM Agents: Threat Surfaces, Attacks, Defenses, and Evaluation. *arXiv preprint arXiv:2606.10749* (2026). https://arxiv.org/abs/2606.10749

[20] Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security 24)*. USENIX Association, Philadelphia, PA, 1831–1847. arXiv:2310.12815. https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

[21] Model Context Protocol. 2025. Model Context Protocol Specification, 2025-11-25 Revision: Authorization. https://modelcontextprotocol.io/specification/2025-11-25/basic/authorization. Accessed 2026-08-27.

[22] Milad Nasr, Nicholas Carlini, Chawin Sitawarin, Sander V. Schulhoff, Jamie Hayes, Michael Ilie, Juliette Pluto, Shuang Song, Harsh Chaudhari, Ilia Shumailov, Abhradeep Guha Thakurta, Kai Yuanqing Xiao, Andreas Terzis, and Florian Tramèr. 2026. The Attacker Moves Second: Stronger Adaptive Attacks Bypass Defenses Against LLM Jailbreaks and Prompt Injections. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1467–1486. https://www.usenix.org/conference/usenixsecurity26/presentation/nasr

[23] Nils Palumbo, Sarthak Choudhary, Jihye Choi, Guy Amir, Prasad Chalasani, and Somesh Jha. 2026. Formal Policy Enforcement for Real-World Agentic Systems. *arXiv preprint arXiv:2602.16708* (2026). https://arxiv.org/abs/2602.16708

[24] Zhenyu Pan, Yiting Zhang, Zhuo Liu, Yolo Yunlong Tang, Zeliang Zhang, Haozheng Luo, Yuwei Han, Jianshu Zhang, Dennis Wu, Hong-Yu Chen, Haoran Lu, Haoyang Fang, Manling Li, Chenliang Xu, Philip S. Yu, and Han Liu. 2025. AdvEvo-MARL: Shaping Internalized Safety through Adversarial Co-Evolution in Multi-Agent Reinforcement Learning. *arXiv preprint arXiv:2510.01586* (2025).

[25] Romualdo Pastor-Satorras and Alessandro Vespignani. 2001. Epidemic Spreading in Scale-Free Networks. *Physical Review Letters* 86, 14 (2001), 3200–3203. doi:10.1103/PhysRevLett.86.3200

[26] Jordi Sabater and Carles Sierra. 2005. Review on Computational Trust and Reputation Models. *Artificial Intelligence Review* 24, 1 (2005), 33–60. doi:10.1007/s10462-004-0041-5

[27] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. 2025. Progent: Securing AI Agents with Privilege Control. *arXiv preprint arXiv:2504.11703* (2025). https://arxiv.org/abs/2504.11703

[28] Hao Song, Yiming Shen, Wenxuan Luo, Leixin Guo, Ting Chen, Jiashui Wang, Beibei Li, Xiaosong Zhang, and Jiachi Chen. 2025. Beyond the Protocol: Unveiling Attack Vectors in the Model Context Protocol (MCP) Ecosystem. *arXiv preprint arXiv:2506.02040* (2025).

[29] Georgios Syros, Evan Rose, Brian Grinstead, Christoph Kerschbaumer, William Robertson, Cristina Nita-Rotaru, and Alina Oprea. 2026. MUZZLE: Adaptive Agentic Red-Teaming of Web Agents Against Indirect Prompt Injection Attacks. In *35th USENIX Security Symposium (USENIX Security 26)*. USENIX Association, Baltimore, MD, 1527–1546. https://www.usenix.org/conference/usenixsecurity26/presentation/syros

[30] Harold Triedman, Rishi Jha, and Vitaly Shmatikov. 2025. Multi-Agent Systems Execute Arbitrary Malicious Code. In *Proceedings of the 2025 Conference on Language Modeling*. https://arxiv.org/abs/2503.12188

[31] Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. 2024. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv preprint arXiv:2404.13208* (2024).

[32] Peiran Wang, Ying Li, and Yuan Tian. 2026. Aligning Provenance with Authorization: A Dual-Graph Defense for LLM Agents. *arXiv preprint arXiv:2605.26497* (2026). https://arxiv.org/abs/2605.26497

[33] Shihao Weng, Yang Feng, Jinrui Zhang, Xiaofei Xie, Jiongchi Yu, and Jia Liu. 2026. ARGUS: Defending LLM Agents Against Context-Aware Prompt Injection. *arXiv preprint arXiv:2605.03378* (2026). https://arxiv.org/abs/2605.03378

[34] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *32nd Network and Distributed System Security Symposium (NDSS 2025)*. arXiv:2403.04960. https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/

[35] Miao Yu, Shilong Wang, Guibin Zhang, Junyuan Mao, Chenlong Yin, Qijiong Liu, Kun Wang, Qingsong Wen, and Yang Wang. 2025. NetSafe: Exploring the Topological Safety of Multi-agent System. In *Findings of the Association for Computational Linguistics: ACL 2025*. 2905–2938. doi:10.18653/v1/2025.findings-acl.150

[36] Lingxi Zhang, Guangtao Zheng, and Hanjie Chen. 2026. When Embedding-Based Defenses Fail: Rethinking Safety in LLM-Based Multi-Agent Systems. *arXiv preprint arXiv:2605.01133* (2026).